

Integral kernels and residue determinants

Exact auxiliary relations and bounded arithmetic certificates

A narrow independently compiled extraction from OpenAI mathematical sources: rational kernel spanning, weighted determinant divisibility and source-bound polynomial relations.

14

general theorem declarations

12

exact worked packets

120

independent matrix
comparisons

Bounded point sets and explicit determinant bounds. Integral lattice saturation, global height bounds and manuscript-wide verification retain separate obligations.

Source, corpus receipts and reproduction commands accompany the repository snapshot.

The mathematical advance

PerfectPower now has an exact bridge from finite arithmetic data to auxiliary polynomial relations. Given integer points and a declared monomial basis, the producer forms the evaluation matrix, constructs integer vectors that annihilate every row, and emits Lean proofs tying those vectors to the literal points and exponents. Separately, a modular row transformation can exhibit a large divisor of a determinant. When an explicit Archimedean bound is smaller than that divisor, the determinant must vanish. The resulting objects are useful in bounded arithmetic searches, local residue clusters, polynomial interpolation and determinant-method experiments. They are not global integral-point classifications.

This extraction is based on OpenAI's public mathematical collection at commit `adc7f1241b42e322a6451854ab7e4b4c146bf78a`. The selected sources are `lean/OAI/NumberTheory/PowerFree/IntegralKernels.lean` and `ResidueRelations.lean`, within family 020 on power-free polynomial values. Their general algebraic arguments are adapted into two independent PerfectPower modules and compiled under this repository's existing Lean 4.20.0 pin. The upstream collection uses Lean 4.34.1. The extraction does not import its full library, upgrade PerfectPower's toolchain, or assert verification of the family-wide density theorem. Apache-2.0 attribution and license are retained in `third_party/openai_math`.

Integer representatives of a rational kernel

Let B be an r by n integer matrix with rationally independent rows. Its Gram matrix $G = BB$ -transpose has positive determinant D . The exact integer matrix $K = D I - B$ -transpose $\text{adj}(G) B$ satisfies $BK = 0$. For any vector x with $Bx = 0$, $Kx = D x$. Since D is nonzero, the columns of K span the entire kernel over the rationals. If B consists of a spanning selection of rows of a larger evaluation matrix A , then $AK = 0$ as well. PerfectPower's generic theorems prove these statements over the appropriate rings and fields, and prove that this construction commutes with a ring homomorphism.

The implementation selects independent rows by exact rational elimination, computes determinants with fraction-free Bareiss elimination, forms the adjugate by exact minors, and removes common factors from nonzero column vectors. It preserves every distinct resulting relation. Rank-zero input yields the identity kernel matrix; full-column-rank input yields no nonzero relations. Dimensions and literal bit lengths are bounded before expensive computation. The computational component is standard-library Python; SymPy is used only for the independent development cross-check.

Rational spanning is not integral saturation. For $B = (1,1,1)$, K has diagonal entries 2 and off-diagonal entries -1. Every integer combination of its columns has all three coordinates congruent modulo 3, whereas $(1,-1,0)$ lies in the integer kernel and violates that congruence. A worked Lean theorem proves this missing vector cannot be an integer column combination. The packet therefore records `integral_saturation_certified = false`. Primitive normalization of individual columns does not remove a joint lattice index. A Smith or Hermite normal-form theorem would be an additional mathematical bridge, not an implicit property of this construction.

Local expansions and determinant divisibility

Suppose each row i of a square matrix has entries divisible by q to the power $w(i)$. Multilinearity of the determinant gives divisibility by q to the sum of all row weights. The extracted generalization allows rows to be linear combinations of weighted local vectors. Expanding the determinant over all selections of local vectors, any repeated selection vanishes by alternation. If every injective selection has weight sum at least m , every remaining term is divisible by q to the power m . This proves `determinant_dvd_of_local_expansion` with explicit coefficient, divisibility and injective-weight premises.

For multivariate polynomial evaluations at coordinates divisible by q , a monomial of total degree e is divisible by q to the power e . The general polynomial-evaluation theorem therefore derives determinant divisibility from a lower bound on the total degree of every injective monomial selection. This theorem supports future residue-patch work. The current Python producer does not automatically derive smooth implicit-function expansions, choose optimal monomial weights, or discharge general geometric dimension bounds.

The executable modular route operates on an arbitrary bounded square integer matrix A . It builds an integral row transformation E and $U = EA$ by modular elimination. Pivot normalization is allowed only for entries invertible modulo the chosen modulus M . Elementary subtraction and row swaps preserve modular invertibility, while each pivot scale is a modular unit. Rows of U divisible by M supply weights zero or one. Their sum gives $D = M$ to the sum of weights. Cancellation requires $\gcd(D, \det(E)) = 1$; the packet supplies enough exact data to check that condition. Lean proves that D divides $\det(A)$ from the transformed row divisibility and this coprimality premise. A composite modulus is supported when all required pivots are units; a nonunit pivot is rejected.

An explicit magnitude bound forces exact vanishing

If the absolute value of every entry in row i is at most $b(i)$, expansion over permutations gives absolute $\det(A)$ at most $n!$ times the product of the $b(i)$. This is intentionally a simple exact bound. If this bound is strictly less than positive D and D divides $\det(A)$, then $\det(A) = 0$. `PerfectPower` proves the row-product bound over the reals and the integer-divisor conclusion in Lean. The bound applies to the original A , even when the transformation E creates much larger intermediate entries.

The worked matrix with rows $(1,2,3)$, $(2,3,4)$, $(3,4,5)$ has an exact magnitude bound of 360. Modular elimination at $M = 1009$ produces a divisible transformed row and a coprime transformation determinant, so 1009 divides $\det(A)$. Since 360 is less than 1009, the determinant is zero. A second worked matrix has two divisible rows at modulus 101, hence divisor 10201. A composite-modulus example uses 35. In contrast, $\text{diag}(3,1)$ at modulus 3 has nonzero determinant 3 and does not meet the strict bound. Its packet remains bound-insufficient; that status does not mean the determinant is nonzero, and cannot be promoted into a native zero theorem.

Source-bound auxiliary polynomials

For points $p(i)$, center z and exponent tuples $e(j)$, the evaluation matrix entry is the product over coordinates t of $(p(i,t)-z(t))$ to the exponent $e(j,t)$. A nonzero integer relation c with $Ac = 0$ defines an auxiliary polynomial vanishing at every supplied point. The native auxiliary emitter separately defines the literal point matrix, center and exponent matrix, proves equality between the supplied evaluation matrix and this expression, and proves each resulting source relation. Concrete nonzero relations and the matrix annihilation identity are checked by kernel reduction. For nonzero rank, the emitted program also connects the literal Gram construction to the reusable rational-spanning theorem.

The corpus contains ordinary and centered parabolas, a Mordell sample on $y^2 = x^3 - 2$, and a signed quartic sample on $y^2 = x^4$. The parabola data recover $y - x^2$ in the declared basis. The corrected Mordell data use $(3,5)$ and $(3,-5)$, with basis $1, x^3, y^2$. The previously listed $(129, \pm 1465)$ are not on this curve and have been removed. The signed quartic includes both ordinate signs and recovers its supplied algebraic relation. These are finite source-bound results. Neither interpolation nor kernel spanning proves that the supplied point set exhausts the corresponding curve.

Public execution and proof boundaries

The JSONL query service now offers `integral_kernel_packet`, `bounded_auxiliary_packet` and `residue_determinant_packet`, matching replay operations `verify_integral_kernel`, `verify_bounded_auxiliary` and `verify_residue_determinant`, and native emitters `native_integral_kernel`, `native_bounded_auxiliary` and `native_residue_determinant`. Every native response returns Lean source and `execution_verified = false`. Execution becomes an established fact only when that particular emitted source is compiled; the release's eleven worked native programs are compiled and audited separately. No theorem here formalizes arbitrary JSON parsing or the Python producer itself.

The computational limits are at most 64 matrix rows, eight columns, four polynomial variables, total monomial degree at most twelve, 256-bit point coordinates and 4096-bit intermediate integers. Native kernel and auxiliary reductions accept at most eight rows, rank at most four, and 64-bit matrix and kernel entries; native determinant reductions require dimension at most four and 64-bit entries. These are resource limits, not mathematical nonexistence conclusions. The producer either returns a bounded object or rejects the requested work; it does not silently truncate the point set, monomial basis or residue data.

Validation and the next bridge

The release contains twelve reproducible packets and eleven native Lean programs, fourteen reusable theorem declarations across the two modules, and a formal integral-index counterexample. Fourteen focused Python tests cover rank extremes, source binding, query operations, malformed data, altered divisibility and magnitude premises, nonunit transformations, composite moduli and unresolved bounds. An independent seeded cross-check compares determinants and ranks against SymPy on 120 integer matrices in dimensions one through six; 31 meet the strict zero bound and no false zero conclusion occurs. The focused audit records exact compilation outcomes and permits only `propext`, `Classical.choice` and `Quot.sound`. Full historical repository verification is not claimed by this focused extraction.

The next high-value connection is a bounded smooth-residue patch: prove Taylor or implicit-function coefficient divisibility for a declared curve and good prime, feed those local vectors into `determinant_dvd_of_local_expansion`, and derive an auxiliary relation covering every point in a declared box and residue class. That would turn the current finite point-set interface into a bounded arithmetic covering theorem. Global Mordell bounds, explicit Matveev premises, complete arbitrary-prime elliptic division and analytic singular-period certification remain independent obligations. The extracted rational kernel matrix can support those algorithms, but does not discharge their missing mathematics.

Reproduction and provenance

Run `make residue-determinant` with the repository's pinned Lean installation and Mathlib cache. The target reproduces the corpus, runs the focused tests, builds both modules, compiles every worked program and checks the axiom outputs. Source files are `PerfectPower/IntegralKernelWitness.lean`, `PerfectPower/ResidueDeterminantCertificate.lean`, `python/perfectpower/residue_determinant.py` and `python/develop_residue_determinant.py`. The handoff is `CLAUDE_CODE_RESIDUE_DETERMINANT_START_HERE.md`; exact receipts and native programs reside in `receipts/residue_determinant`. The source collection is <https://github.com/openai/math/tree/ad7f1241b42e322a6451854ab7e4b4c146bf78a> and its family description is `lean/docs/020.md`. The complete source archive preserves concurrent repository work and is split into parts below 30 MB.